EXPOSITORY ARTICLE ▪ BASED ON EXACT ADDRESSING, PRELIMINARY RELEASE 4.14

Two Geometries of Automated Theorem Proving

Histories, Proofs, Certificates, and Safe Compression

<Brian Tenneson>

October 30, 2026 • ATLAS-reviewed expository edition 0.2

See the AI disclaimer that follows this essay.

Abstract. What does an automated theorem prover actually traverse when it searches for a proof? Its operational history is not the same thing as the proof it may eventually produce. One graph records possible computational journeys, including unsuccessful ones; another organizes verified derivations of a common theorem. A third structure mediates between them: a checker-accepted certificate extracted from a recorded computation. This article explains four results from the prologue of *Exact Addressing* 4.14: rooted history unfolding (P.1), reversal and safe root-gluing of annotated proof DAGs (P.2), conditional certificate extraction (P.3), and cost-preserving operational quotients under strong matching conditions (P.4). The perspective exposes a research problem: when, if ever, can safe compression reduce the *fully charged* cost of finding a certified proof? The answer is not supplied by any of the four structural results alone.

Reader's guide. No specialized knowledge of proof assistants is required, but familiarity with graphs and basic logical inference will help. Formal conditions are given where the distinction between a theorem and an aspiration matters. New toy examples are explanatory illustrations, not additional results attributed to the source.

1 One prover, two different maps

Suppose a theorem prover begins with a conjecture, chooses an inference, tries an auxiliary fact, revises its search, and ultimately prints a proof. An informal description may call the whole process a "proof search graph." But this phrase hides two genuinely different maps.

The *map of histories* contains the computations the machine is allowed to carry out. Its vertices represent complete finite runs from a common starting point; its arrows append legal events. It includes finite failed runs, finite prefixes of runs that may continue indefinitely, and searches that have not yielded a certificate. The *map of proofs* contains accepted proof objects: finite, annotated derivations that meet the checker and hypothesis requirements of a fixed formal system. Several different proof objects may establish the same conclusion, and a single proof can be discovered, retrieved, or repeated along several different histories.

The question is not which map is "really" the proof. They answer different questions. A history tells us *how an execution proceeded*; a proof object tells us *why a conclusion follows under its stated formal rules*. A trusted bridge between these questions requires more than a resemblance between their diagrams.

This article uses the same four labels P.1–P.4 as the original prologue. They are not steps in a universal implication chain. Each rests on its own declared model and, in two cases, on additional

effective or checking assumptions.

2 Unfolding a search without losing its past: P.1

2.1 Why the operational state graph is not a history tree

Write an automated prover's operational model as a labeled transition system

$$A = (S, s_0, M, \longrightarrow, \mathrm{Obs}), \qquad s \xrightarrow{a} s',$$

where S contains the represented complete states, s_0 is the start, M supplies event labels, and Obs records the declared observations. "Complete" is relative to what the model intends to preserve: queue positions, search budgets, scheduler choices, authorization, checker scope, receipts and modeled random or environmental outcomes may all matter. If distinct events are intended to remain distinguishable, they must have distinct represented labels or identifiers.

Operational states may repeat. After inspecting a candidate, a program may return to the same queue configuration. A cycle in the *state graph* is therefore ordinary. It would be a mistake, however, to collapse the two visits into one *history*. Their pasts need not agree.

A finite legal history is the full sequence

$$h = (s_0, a_1, s_1, \ldots, a_k, s_k), \qquad s_{i-1} \xrightarrow{a_i} s_i,$$

including the empty-step history $\varepsilon = (s_0)$. The history unfolding $\mathcal{H}(A, s_0)$ has one vertex for each such sequence; one history points to another when exactly one legal step has been appended. Its endpoint map $e(h) = s_k$ forgets the past and retains the last state.

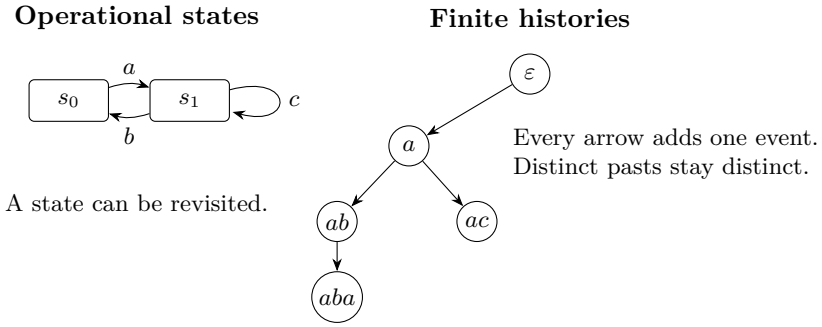


Figure 1: A state-space cycle disappears as a cycle when we unfold runs into distinct finite histories. The letters abbreviate event sequences; the picture is a finite fragment of a possibly infinite unfolding.

2.2 What P.1 actually guarantees

Structural statement (P.1, expository formulation)**.** *For a fixed represented transition system, the finite-history unfolding is a rooted directed tree with root ε. Every history has exactly one path back to that root. For any finite collection K of histories, $\{\varepsilon\}$ together with all prefixes of members of K forms a finite rooted tree (even when $K = \varnothing$), the union of all such prefix trees is the full unfolding, and the endpoint map covers exactly the states reachable by finite legal runs.*

The argument is simple but decisive. Any non-root history has one immediate predecessor: delete its final labeled event. Repeating this deletion reaches ε. Since each forward edge increases the length of a history, no directed cycle can appear. A finite family of finite sequences has only finitely many prefixes; including ε covers even the empty collection. Every legal history belongs to the prefix tree generated by itself.

The conclusion says *all finite modeled histories are represented*, not that a finite program has listed them. The tree may be infinite, its legal-step relation may be difficult or impossible to decide effectively, and history representation need not be economical. Nor can this construction recover events omitted from the original state model: two distinct physical events with the same represented state-label-state triple are identified unless distinct event identifiers were recorded. These limitations matter when someone proposes a memory-saving or search-speedup algorithm based merely on the existence of the tree.

3 Many proofs, one conclusion: P.2

A proof object is different from a run. Fix a formal calculus F, a hypothesis scope Γ, and an independent checker V. Let $\mathsf{P}_{F,\Gamma,V}$ denote the accepted finite *terminally marked annotated proof DAGs*. Such a directed acyclic graph retains occurrences of formulas, rule-application nodes, ordered premise ports, assumptions, and checker/certificate annotations. It has one designated terminal conclusion occurrence; every retained vertex contributes along a directed path to that occurrence.

If $\tau(D)$ is the conclusion formula of proof D, then for a formula T the *coterminal fiber* is

$$\mathcal{D}_T = \{D \in \mathsf{P}_{F,\Gamma,V} : \tau(D) = T\}.$$

This groups proofs by their common conclusion without pretending that distinct copies of that conclusion, or of any intermediate formula, are identical vertices. Two proofs of T may share formula labels and still be different proof objects.

3.1 Reversing proof edges does not reverse logic

In the usual forward orientation, premise occurrences and inference rules point toward the conclusion. Reverse every edge and the designated conclusion becomes a root from which backward *obligations* can be read. It is tempting to interpret the new arrows as instructions to derive premises from the theorem. That would be incorrect. The arrows are reversed *dependencies*; the inference rules themselves have not been inverted.

For example, if a legal rule proves T using both P and Q, looking backward from T requires satisfying *both* obligations. These are an AND pair. If two complete rule applications offer different ways to prove T, those alternatives form an OR choice. Preserving rule-instance and premise-port annotations is what lets the diagram retain this distinction.

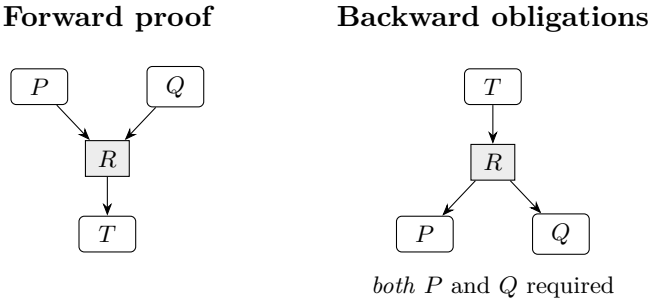


Figure 2: Reversing edges changes the reading direction, not the licensed inference. Rule R still requires both premises; the reversed picture is an obligation structure, not a new proof rule.

Structural statement (P.2, expository formulation)**.** *Reversing all edges of a finite annotated, terminally marked proof DAG preserves acyclicity, turns its designated conclusion into a root from which all retained vertices are reachable, and is an involution:* $(D^{\mathrm{op}})^{\mathrm{op}} = D$. *For finitely many proofs with*

the same conclusion, taking disjoint tagged reversed copies and identifying only *the designated roots produces an acyclic common-root graph. The proof-instance tags must remain if the components are to be recoverable. This construction does not authorize identifying any other vertices by their formula labels.*

Why only the roots? If two acyclic proofs are glued merely because they contain the same formula symbols, their local dependence structures may create a cycle together. The original prologue gives a compact example. With assumptions A and B and licensed one-premise rules $A \Rightarrow B$, $B \Rightarrow A$, $A \Rightarrow T$ and $B \Rightarrow T$, one proof may use $A \to B \to T$, and another $B \to A \to T$. Their reversals contain $T \to B \to A$ and $T \to A \to B$. Identifying the distinct A and B occurrences across the two graphs creates $A \leftrightarrow B$, a directed cycle. This is not a problem with either original proof. It is a problem introduced by an unjustified quotient of their occurrences.

To see why root-only gluing is safe, note that each reversed root has no incoming edge. A cycle visiting two tagged components would have to enter their shared root, which no edge can do; a cycle within a single component is already excluded by acyclicity. Thus the coterminal family is a useful way to organize alternatives, but *sharing* internal nodes demands a further preservation argument. An economical representation of proof diversity cannot be designed by formula-label equality alone.

4 From a computation to a certificate: P.3

The history tree and the collection of verified proof DAGs do not automatically speak to each other. A third object is needed: an explicit *extractor*. Let $\mathcal{H}_{\mathrm{cert}} \subseteq \mathcal{H}(A, s_0)$ be a declared domain of histories from which a partial computable map E successfully extracts a candidate proof object. Suppose the independent checker accepts the result in a fixed scope:

$$E : \mathcal{H}_{\mathrm{cert}} \longrightarrow \mathsf{P}_{F,\Gamma,V}, \qquad V(E(h), \tau(E(h)), \Gamma) = \mathsf{accept}.$$

This is a *typed* map: the input is an execution history and the output is a proof DAG, not just an arbitrary string or an unexplained claim of success.

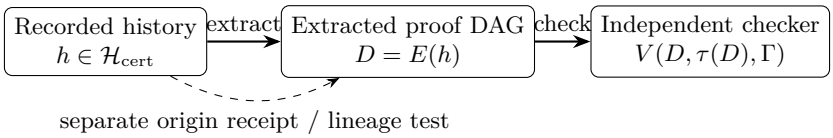


Figure 3: Checker acceptance and checkable provenance are different assertions. The dashed relation needs its own evidence, even if the extracted proof passes the checker.

4.1 Validity is not a receipt of origin

Consider a prover that has spent an hour exploring a conjecture. At the end, a cache lookup returns a pre-existing correct proof. A checker may validate the retrieved object perfectly. But the hour-long search did not thereby *produce* that proof. The validity of the object and its origin in this specific history are separate facts.

The stronger claim can be expressed by a checkable relation $\mathsf{Origin}(h, D)$ that links the proof to ledger events, proof-object hashes, authority and checker scope, or other promised lineage data. If the declared origin check succeeds, the result is provenance-preserving *to the extent the relation establishes.* It does not follow that the program discovered the proof from scratch; that depends on what the receipt actually attests.

Conditional correspondence (P.3, expository formulation)**.** *Every history in the declared extraction domain yields an accepted proof DAG belonging to exactly one conclusion fiber $\mathcal{D}_{\tau(E(h))}$. If the checker*

is sound for the fixed calculus and assumptions, acceptance entails the corresponding formal derivability. A further provenance conclusion requires a separately satisfied origin condition. Surjectivity of E requires proof realization*: every accepted DAG must arise as the extraction of at least one legal finite history. Injectivity is not implied.*

The proposition follows directly from the typed domain and conclusion map; the additional claims use, respectively, checker soundness, the separate origin test, and the definition of surjectivity. Each clause answers a different question. A sound checker helps establish *is this a valid proof?* An origin receipt addresses *where did this proof come from?* Surjectivity asks *can the allowed execution system realize every accepted proof?* And injectivity asks *could two different histories produce the same proof?* There is no reason to expect that last property in ordinary search.

5 Compression without counterfeit paths: P.4

Now change the object being compressed. Instead of merging proof occurrences, suppose we want to merge *operational states*. This could make a search space smaller, but it could also introduce abstract paths that no concrete machine run can follow.

Take a reachable state system with nonnegative primitive edge costs, writing $s \xrightarrow{a,c} t$. Fix a target set G and an equivalence relation $\sim$ on reachable states. The quotient represents each class $[s]$ as one abstract vertex. An existential quotient includes an abstract edge $[s] \xrightarrow{a,c}_Q [t]$ whenever *some* representatives realize an edge with that label and cost.

An existential definition alone is unsafe for path planning from a specified representative. A class may contain one state with a useful next step and another state without it; chaining class-level edges can then depict a path that switches representatives impossibly between steps. The repair is a strong matching condition.

5.1 Matching what matters

The original theorem separates three requirements:

1. **Target and observation invariance.** Equivalent states agree about membership in G and about any protected observations.
2. **Two-way label-and-cost bisimulation.** If $s \sim t$, every transition $s \xrightarrow{a,c} s'$ has a matching transition $t \xrightarrow{a,c} t'$ with $s' \sim t'$, and vice versa. Both event label and primitive cost must match.
3. **Effective lifting and receipts, when claimed.** An effective `Lift` procedure must implement each abstract step from each compatible reachable concrete representative. Certificate transport needs its own scope-preserving receipt interface and an independent check.

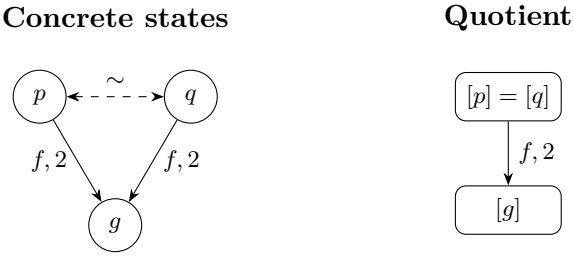


Figure 4: A toy matching step with identical labels and primitive costs. This fragment illustrates what a bisimulation condition checks; complete bisimulation requires *all* outgoing transitions to match.

Conditional preservation (P.4, expository formulation). *Under target/observation invariance and two-way label-and-cost bisimulation, an original state s and its quotient class $[s]$ have exactly the same finite target-reaching sequences of transition labels and primitive costs. Consequently the sets of summed primitive path costs, and their infima, coincide. The infimum is $+\infty$ when no target path exists. Effective lifting follows only when the additional implementation interface is available; a valid original certificate follows only when the separate receipt translation returns an independently accepted object.*

The proof is a path-matching argument. Any concrete path projects to a quotient path. Conversely, at each abstract step the bisimulation condition supplies an edge from the *particular* concrete representative that has actually been reached. Induction lifts any finite quotient path while preserving labels, costs and whether the final state is a target. For computable lifting, however, an existence proof must be supplemented by an algorithm that can find the matching successor.

Here is an elementary caution. Suppose the initial state u has distinct legal steps $u \xrightarrow{a} p$ and $u \xrightarrow{b} q$, and q (but not p) has a `finish` step to a target state g. Both p and q are reachable. If $p \sim q$ is declared anyway, the existential quotient displays a two-step trace $[u] \xrightarrow{a} [p] = [q] \xrightarrow{\texttt{finish}} [g]$ that cannot lift from u: the a step ends at p, where `finish` is disabled. The bisimulation hypothesis fails. Conversely, if p and q offer exactly matching labeled and costed behavior (including every loop and alternate branch), their merger can be safe for primitive traces. Neither observation says the quotient is computationally worthwhile.

6 Three kinds of address, not one

An *exact address* is an injective effective identity code for a specified finite object. The object must be named before the address makes sense. A complete history, a certified proof DAG, and a quotient class are different types of things and need not have compatible codes.

Where suitable encodings exist, the distinctions look like

$$\underbrace{\mathsf{addr}_H(h)}_{\text{history identity}}, \qquad \underbrace{\mathsf{addr}_P(E(h))}_{\text{proof identity, } h\in\mathcal{H}_{\text{cert}}}, \qquad \underbrace{\mathsf{addr}_Q([e(h)])}_{\text{class identity, if effective}}.$$

The first names a computation's full represented past. The second names an accepted proof extracted from a qualifying history. The third names a class of the history's endpoint under a chosen operational quotient. No single expression should silently be substituted for the others.

An address is not itself a theorem of locality. Neighboring integers used as codes need not describe nearby proofs; Even a collision-free address code need not preserve logical dependency or the cost of reaching a target; an ordinary fixed-length hash, moreover, is not injective on an unbounded collection of finite objects. Even if a state equivalence is mathematically sound, efficiently obtaining a canonical representative of each class is a separate computational problem. A computable enumeration of states together with a decidable equivalence relation provides one route to canonical class coding; it does not automatically make the resulting address system fast or small.

This point is particularly important in the setting of *exact addressing*: a clean indexing discipline can support auditing and navigation while leaving the search-complexity question entirely open.

7 Where the true economic question begins

The easiest cost to assign in a graph is the sum of weights along a particular path. But a theorem prover does not magically receive the successful path in advance. It must decide which branches to inspect, perhaps construct a quotient, schedule tasks, call a checker, store certificates, and reconstruct whatever its compressed representation has omitted.

Thus *primitive path cost* and *fully charged discovery cost* are different measurements. P.4 preserves the former under strict hypotheses. It says nothing unconditional about the latter.

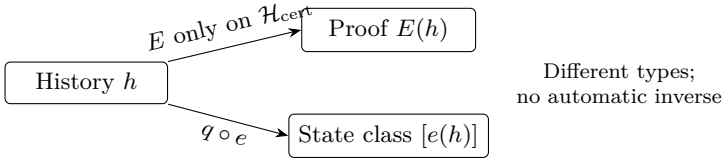


Figure 5: A certified history can have a proof address and a quotient-class address, but the underlying objects and the information they retain are not interchangeable.

Quantity	What is being counted?
Primitive path cost	The sum of declared weights for steps along one finite legal execution path.
Representation overhead	Quotient construction, data structures, addresses, canonicalization and lookup.
Search overhead	Which successor to examine, policy updates, branching, restarts and scheduling.
Certification overhead	Verifier calls, receipt storage, proof-object extraction and reconstruction.
Fully charged discovery cost	The declared aggregate cost of the whole certified discovery process against a frozen baseline.

Table 1: The cost ledger that prevents a path-preservation theorem from being mistaken for a performance theorem. The exact cost model must be fixed before comparison.

Imagine reducing a state graph from a million vertices to a thousand equivalence classes. This is a dramatic representational contraction. Yet suppose computing class addresses is very expensive and each abstract move requires costly representative reconstruction. The compressed solver may lose to the original. Conversely, a modest, effectively navigable quotient that maintains cheap certificate transport could be useful, provided a matched experiment or theorem accounts for its expenses. Graph size alone does not settle the comparison.

The capstone question posed in the prologue can therefore be stated without ambiguity:

> *Can one construct nontrivial, effectively navigable representations of histories, proof obligations or operational states that preserve the intended certified behavior and actually reduce the fully charged expected or worst-case cost of finding accepted proofs, relative to a fixed baseline?*

A credible positive answer needs more than an elegant picture: it needs declared targets, verifier, input distribution when expectations are used, search policy, resource budget, costs of construction and navigation, and checkable lifting of proofs and receipts. The present article does not supply such an answer.

8 The synthesis: a typed architecture for search and knowledge

The four results form a useful architecture precisely because they remain distinct. P.1 tells us how to represent every finite *modeled execution* without confusing a state with its history. P.2 tells us how to organize multiple *accepted derivations* of one conclusion without merging their internal occurrences recklessly. P.3 specifies the conditions under which a history can be associated with a *checked proof object*, while keeping validity separate from checkable provenance. P.4 explains when collapsing operational states retains the primitive traces that a search process may need.

One can picture three layers: executions, certificates, and equivalence classes. Between them are maps, each carrying a specific obligation. The endpoint map forgets past events. The extractor produces an accepted object only on a declared domain. The quotient map forgets distinctions only under a declared equivalence. An origin receipt, if offered, is a fourth piece of evidence; it is not supplied automatically by any of those maps.

The architecture suggests three questions to ask of any new proof-search representation. *What information was forgotten? Which correctness claim survives the forgetting, and by what proof or independent check? What does it cost to search and reconstruct once the information is forgotten?* The first is a representation question, the second a mathematical or verification question, and the third an economic question. Treating them as separate allows their answers eventually to reinforce one another.

9 A short glossary and statement of limits

History unfolding. A rooted tree of all represented finite legal executions, not a claim that all such histories have been physically enumerated.

Coterminal proof family. Accepted annotated proof DAGs sharing a designated conclusion formula, not a single graph with formula-identical vertices forcibly merged.

Backward obligation. A dependency viewed from conclusion toward prerequisites; it is not the inverse of a legal inference rule.

Certified extraction. A specified partial history-to-proof mapping whose outputs are accepted by a fixed independent checker.

Provenance receipt. Additional checkable evidence connecting an accepted proof to its alleged execution origin.

Bisimulation. A relation in which matching states can reproduce one another's relevant labeled transitions; P.4 also demands exact matching of primitive costs.

Exact address. An effective injective identifier for a given type of finite object; no semantic proximity or efficiency guarantee is implied.

This article is an exposition of a specific prologue, not an independent formal verification. The roots of the unravelling and bisimulation methods predate this synthesis, as the source itself acknowledges. In particular, P.1 does not ensure practical enumeration; P.2 does not license unrestricted formula-label merging; P.3 does not identify proof validity with proof provenance; and P.4 does not imply faster fully charged proof discovery. These negative boundaries are part of the content, not qualifications added after the fact.

Source, references, and editorial provenance

The mathematical backbone and the labels P.1–P.4 are adapted from <Brian Tenneson>, *Exact Addressing*, Preliminary Release 4.14, Prologue, pp. 8–13 [1]. In the source, these results are separately conditioned and do not form a universal implication chain. This article reorganizes the explanation for an expository audience; its small counterexamples, diagrams, and cost table serve as illustrations rather than independently certified theorems. Earlier work on transition-system unravelling and bisimulation is cited below, not presented as original to this adaptation [2, 3].

References

[1] <Brian Tenneson>, *Exact Addressing, Proof Geometry, Locality, Verifier-Safe Hilbert Search, and Formalized Scientific Hypothesis Generation*, Preliminary Release 4.14, Prologue, pp. 8–13, October

23, 2026. Author's source manuscript; the present article is an expository adaptation.

[2] Y. Venema, *Lectures on the Modal μ-Calculus*, 2024 notes, Definition 1.23 (path unravelling). https://tsinghualogic.net/JRC/wp-content/uploads/2024/07/modal-mu-calculus.pdf.

[3] A. Joyal, M. Nielsen, and G. Winskel, "Bisimulation from Open Maps," *BRICS Report Series*, vol. 1, no. 7, May 1994. https://doi.org/10.7146/brics.v1i7.21663.

Disclaimer on Mathematical Accuracy, Authorship, and Artificial Intelligence

Every reasonable effort has been made to ensure that the mathematical exposition presented herein is factual, correct, logically sound, sufficiently complete in its treatment of relevant facts (not in the technical sense of Gödelian completeness), and supported by rigorous proofs or, at minimum, not readily susceptible to refutation. Nevertheless, no guarantee of correctness, completeness, consistency, or provability is implied. The authorship and composition of this work involve a combination of human and artificial intelligence: an AI determines the placement of some statements, but not all statements, and the placement of some individual characters, but not all characters. Generally, the AI involved is either DATA MIND or CAMI, both of which have been personally augmented by <Brian Tenneson> with structured, automated-theorem-proving (ATP) forms of creativity and intelligence. Mathematical development, organization, formulation, and revision may therefore reflect varying degrees of human and AI contribution, without implying that either party is solely responsible for any particular passage. ATLAS is employed as an additional mathematical and editorial auditor, alongside another AI reviewer and the author's own examination, to identify errors, omissions, unsupported assertions, and opportunities for improvement. Such review is not equivalent to independent formal verification of every claim. Responsibility for the decision to present and publish the resulting work remains with the human author.

Review-status clarification. An in-session ATLAS-style source-concordance, mathematical-exposition, and layout audit was performed for this revised article. The review does not constitute a separate independent auditor, an external peer review, or a proof-assistant verification of P.1–P.4 or of the underlying source manuscript.